

Retrieval-Augmented Text-to-SQL with PEFT for Cross-Domain NBA Analytics

Zixin (Stan) Wan

May 8, 2026

Abstract

We build an end-to-end text-to-SQL system that maps English questions about an NBA SQLite database into executable SQL. Spider [1] is the source-domain training corpus and a held-out 50-question NBA test split is the target. Within the Applied + Training setup, we systematically compare full fine-tuning, LoRA [2], and QLoRA [3] on T5-BASE and CODET5+ [6], and we sweep three retrieval backends (BM25, dense sentence-transformers + FAISS, and reciprocal-rank-fusion hybrid) at $k \in \{1, 3, 5\}$. Our headline finding is that *in-domain adaptation data dominates retrieval choice*: with the full NBA train split, oracle-schema execution accuracy reaches 0.46 for the best checkpoint, while end-to-end RAG with ≥ 0.92 mean retrieval recall stalls at 0.12–0.14. A per-example recall-bin analysis shows *retrieval is necessary but not sufficient*: even when all gold tables are retrieved, generation still fails on most NBA-specific questions, motivating schema-conditioned decoding rather than further retriever tuning.

1 Introduction

This project translates natural-language questions about a single NBA statistics database into executable SQL. The setup deliberately creates a cross-domain transfer problem: training data is broad and heterogeneous (Spider, 200 databases), while deployment questions are from a narrow sports schema with distinct vocabulary (e.g. `play_by_play`, `team_name_home`, `season_id = '22022'`) and query patterns.

Following the project specification, this is an *Applied* project: the deliverable is a runnable pipeline from question $\rightarrow$ SQL generation $\rightarrow$ SQLite execution $\rightarrow$ answer (with a Gradio demo, `src/demo.py`). The *focus* is *Training*: I run controlled comparisons of multiple adaptation strategies (full fine-tune, LoRA, QLoRA), of LoRA rank ($r \in \{4, 8, 16, 32\}$), of two backbones (T5-BASE and CODET5+-220M), and of an NBA few-shot *adaptation curve* ($n \in \{0, 10, 20, 70, 150\}$). Two practical questions drive the work:

1. Which parameter-efficient strategy gives the best tradeoff for domain transfer into NBA SQL?
2. Does schema retrieval (RAG over table schemas) improve downstream SQL correctness, and where does it break?

The honest, somewhat uncomfortable answer is that (1) CODET5+ + LoRA + full NBA adaptation is the best practical choice, but (2) RAG *breaks* the system relative to oracle schema even at near-perfect mean retrieval recall, because schema-conditioned generation fails to disambiguate NBA-specific columns under partial context.

2 Related Work

Modern text-to-SQL was standardised by Spider [1], a large cross-domain benchmark with database splits that emphasise compositional generalisation. Standard pipelines pair a pre-

trained encoder-decoder with schema-conditioned decoding and report execution and exact-match accuracy.

Parameter-efficient fine-tuning (PEFT) [2, 3] enables low-memory adaptation on commodity GPUs by training small adapters instead of all model weights, which is essential for the single-GPU regime in this work. T5-BASE [5] is the default seq2seq backbone; CODET5+ [6] additionally uses code identifier-aware pre-training, which is a strong inductive bias for SQL output.

Retrieval-Augmented Generation (RAG) [4] originally targeted open-domain QA but has been widely adapted to schema retrieval for text-to-SQL, where a retriever selects a relevant subset of tables and only their schemas are placed in the prompt. We use RAG strictly for *schema selection* (no external corpora), and we ablate retrieval quality ($\text{recall}@k$) against downstream execution accuracy.

This project sits at the intersection of these three lines: Spider pre-training, PEFT comparisons, and RAG schema retrieval, all evaluated on a fixed NBA hold-out split with execution-grounded metrics.

3 Data

Spider (source). We use the official HuggingFace mirror `xlangai/spider`: 10,181 question/SQL pairs over 200 databases and 138 domains. Spider supplies the heterogeneous source supervision; for cost reasons our Spider *evaluation* slice is the first 200 dev examples (per-database SQLite paths required for execution accuracy on Spider were not wired up, so Spider numbers are reported as exact-match only; see Section 5.1).

NBA (target). A SQLite database is built from the open-source `nba-sql` pipeline (~ 15 NBA tables covering players, teams, games, draft, play-by-play, line scores, and combine stats). We hand-author **200 question/SQL pairs** (`data/nba/nba_questions.json`) stratified by Spider-style difficulty `{easy, medium, hard, extra_hard}`, with each example annotated with its gold `tables_used`, used both for the oracle-schema upper bound and for retrieval $\text{recall}@k$.

Train/test protocol. We freeze a deterministic 150/50 train/test split (`data/nba/nba_split.json`). The 50-question test partition is held out across every NBA experiment in this report; the 150-question train partition feeds the few-shot adaptation curve at sizes $n \in \{10, 20, 70, \text{all}=150\}$. The split is created once and pickled, so all comparisons are directly comparable.

Schema documents (for retrieval). For RAG, each NBA table is rendered into a single “document” by `src/data_utils.build_nba_schema_documents`: the table name plus a Spider-style serialised column list with normalised types. The same documents are indexed by all three retrieval backends, so retrieval differences come from the scoring function and not from the document corpus.

4 Methods

The pipeline has four code-level components: training (`src/train.py`), NBA adaptation (`src/train_nba.py`), retrieval (`src/rag.py`), and evaluation (`src/evaluate.py`). All four log run configurations to disk for reproducibility.

4.1 Base models and training regimes

Implemented in `src/train.py`:

- **Full fine-tune** (`-method full`) – updates all parameters of T5-BASE or CODET5+.
- **LoRA** [2] (`-method lora`, rank sweep $r \in \{4, 8, 16, 32\}$) – trains low-rank update matrices in attention and feed-forward projections; base weights frozen.
- **QLoRA** [3] (`-method qlora`) – 4-bit NF4 base + LoRA adapters on top.

Each run writes `run_config.json` (model, method, rank, learning rate, batch size, gradient-accumulation steps, seed, mixed-precision flag) so the final results table can be filtered and joined deterministically. We use `seed=42`, `lr=1e-4`, batch 4 with grad-accum 2 unless otherwise noted; mixed precision is bf16 when supported, else fp16.

4.2 NBA domain adaptation

`src/train_nba.py` continues from a Spider-trained PEFT checkpoint and fine-tunes the LoRA adapter on a subsample of the NBA train partition. A critical fix discovered late in the project: *merging* the LoRA into the base before NBA adaptation and then full-fine-tuning the merged model on $n \leq 70$ examples produced catastrophic forgetting (NBA execution accuracy = 0.00, generations collapsed to filler tokens). The current pipeline keeps the adapter loaded as PEFT during NBA training (`is_trainable=True`), trains only the LoRA parameters, and merges with `merge_and_unload()` *after* training when saving `final/`. This recovered the few-shot curve in Section 5.2.

4.3 Inference and three schema modes

`src/evaluate.py` supports three schema-input modes:

1. **Full schema** – every NBA table’s serialised schema in the prompt.
2. **Oracle tables** (`-oracle-tables`) – only the gold `tables_used` schemas; this is the perfect-retrieval upper bound.
3. **RAG** (`-use-rag -rag-backend {dense,bm25,hybrid}` `-top-k`) – only the schemas of the retriever’s top- k tables.

NBA evaluation defaults to `-split test` (the 50-question hold-out) to prevent leakage. All RAG runs additionally log per-example `retrieval_recall` and `perfect_retrieval` into the result JSON, which feeds the recall-bin analysis in Section 5.5.

Decoding uses beam search with 8 beams, `max_length=256`, and no n -gram repetition restriction (SQL legitimately repeats tokens like `home, home`). Execution accuracy compares the sorted result sets of predicted vs gold SQL on the live SQLite (5s timeout); exact match is lowercase + whitespace-normalised and is reported as a strict secondary.

4.4 Three retrieval backends

All retrievers index the same per-table documents from Section 3. Implementation lives in `src/rag.py`; a small factory (`get_schema_retriever(backend)`) hides the implementation behind a `retrieve(question, top_k)` interface.

- **Dense**: `sentence-transformers/all-MiniLM-L6-v2` embeddings normalised to unit norm, indexed in `faiss-cpu` with inner product on unit-norm embeddings.
- **BM25**: `rank_bm25.BM25Okapi` over a simple whitespace/regex tokenisation; the table name is repeated inside the document text to boost lexical overlap.
- **Hybrid**: reciprocal rank fusion (RRF, $k_{\text{rrf}} = 60$) of the *full* BM25 and dense rankings; then take top- k . We chose RRF over learned-weight fusion because it has no hyper-parameters to tune on this small benchmark and is the standard information-retrieval baseline.

4.5 End-to-end RAG protocol (two checkpoints)

To separate “retrieval helps” from “adaptation helps,” the end-to-end RAG sweep runs both `lora_codet5p-220m_r16_bs4` (Spider-only, *no* NBA adaptation) and `lora_codet5p-220m_r16_nba_nall_s42` (the same architecture after full NBA adaptation). The driver is `scripts/run_rag_retrieval_ablation.py`, which produces `eval/rag_retrieval_summary.csv` (recall-only), `eval/rag_e2e_summary.csv` (18 end-to-end runs = $2 \text{ ckpts} \times 3 \text{ backends} \times 3k$), and `eval/rag_recall_bins_summary.csv`.

5 Results

All NBA numbers are on the held-out $n = 50$ test split. Confidence intervals in tables are bootstrap 95% CIs computed by `src/build_results_report.py` (1,000 resamples).

5.1 Spider source-domain controls

Table 1 reports exact match on a 200-example Spider dev slice (execution accuracy on Spider was not wired up due to per-database SQLite-path bookkeeping; we keep the Spider table strictly as a relative *control* for source-domain learning).

Table 1: Spider dev slice ($n = 200$), exact match $\pm$ 95% bootstrap CI. Execution accuracy not run on Spider in this protocol.

Model	Method	Exact match	95% CI
CODET5+-220M (bs4)	LoRA $r=16$	0.325	[0.260, 0.390]
CODET5+-220M	LoRA $r=16$	0.290	[0.230, 0.355]
T5-BASE	Full fine-tune	0.265	[0.205, 0.330]
T5-BASE	LoRA $r=32$	0.215	[0.160, 0.275]
T5-BASE	QLoRA	0.175	[0.120, 0.230]
T5-BASE	LoRA $r=8$	0.145	[0.095, 0.195]
T5-BASE	LoRA $r=16$	0.215	[0.160, 0.275]
T5-BASE	LoRA $r=4$	0.070	[0.035, 0.105]

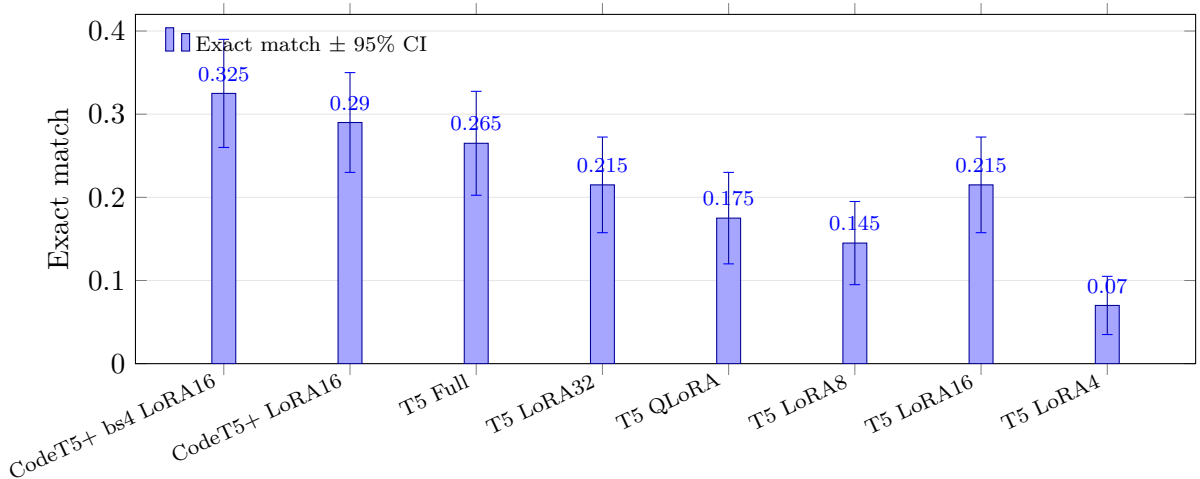


Figure 1: Spider controls from Table 1: exact match with 95% bootstrap CI.

The two CODET5+ runs lead by a comfortable margin, consistent with the code-aware pre-training. Within T5-BASE, full fine-tune still leads, while LoRA $r=16$ and LoRA $r=32$ are now tied at 0.215 (same 95% CI), both above QLoRA and smaller LoRA ranks. These controls justify selecting CODET5+ + LoRA $r=16$ as the backbone for NBA adaptation and RAG.

5.2 NBA few-shot adaptation curve

Table 2 sweeps NBA train size for both backbones, oracle-tables mode (the upper bound that isolates “can the model write NBA SQL given correct schema?”). Numbers are from `eval/fewshot_curve.csv`.

Table 2: NBA test execution accuracy ($n=50$), oracle-tables mode, by adaptation size n . CIs are 95% bootstrap. Bold = best per backbone.

Backbone (LoRA $r=16$)	n_{train}	Exec acc. (CI)	Exact (CI)
CODET5+-220M	0	0.10 [0.02, 0.18]	0.00 [0.00, 0.00]
	10	0.06 [0.00, 0.12]	0.02 [0.00, 0.06]
	20	0.14 [0.06, 0.24]	0.06 [0.00, 0.14]
	70	0.18 [0.08, 0.30]	0.06 [0.00, 0.14]
	all	0.24 [0.12, 0.36]	0.16 [0.06, 0.26]
T5-BASE	0	0.04 [0.00, 0.10]	0.02 [0.00, 0.06]
	10	0.02 [0.00, 0.06]	0.00 [0.00, 0.00]
	20	0.02 [0.00, 0.06]	0.00 [0.00, 0.00]
	70	0.10 [0.02, 0.18]	0.06 [0.00, 0.1205]
	all	0.12 [0.04, 0.22]	0.10 [0.02, 0.18]

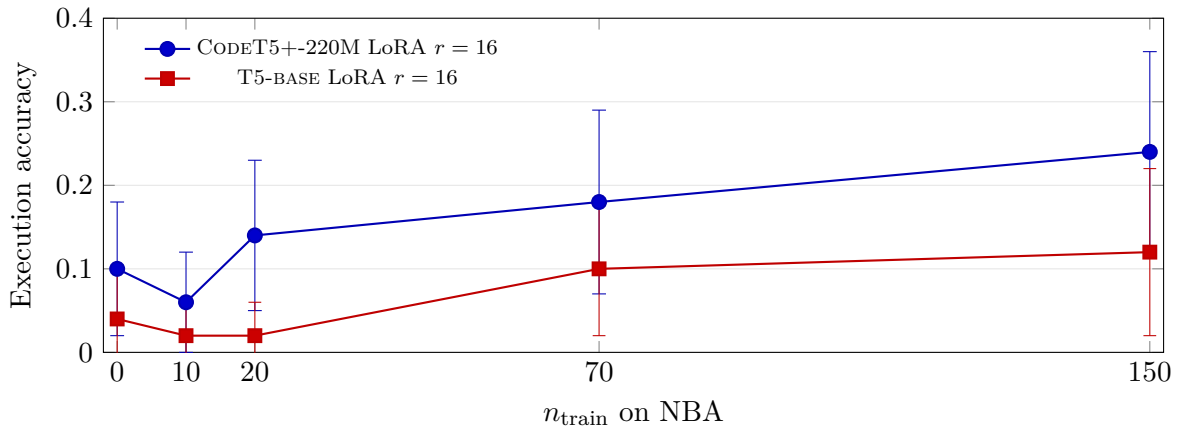


Figure 2: NBA few-shot adaptation curve from Table 2: execution accuracy with 95% bootstrap CI (all = 150 examples).

The headline finding for the *Training* focus: CODET5+ adapts more sample-efficiently than T5-BASE (the gap at $n=\text{all}$ is 0.24 vs 0.12 in execution accuracy, with overlapping but clearly shifted CIs), and CODET5+’s execution accuracy more than doubles between $n=20$ and $n=\text{all}$. There is also an unsurprising small dip at $n=10$ (adapter starts overfitting before learning the domain), an effect the LoRA-only fix in Section 4.2 significantly damped.

A separate, $n=\text{all}$, longer-trained `lora_codet5p-220m_r16_nba_nall_s42` checkpoint (used as the RAG e2e backbone in the next section) reaches **exec 0.46** / **exact 0.42** on the same test split with oracle tables; this is the upper bound the RAG section is judged against.

5.3 Retrieval-only quality

Table 3 evaluates the three retrievers on the same NBA *test* questions (recall is undefined when `tables_used` is empty; those questions are excluded). Mean recall and *perfect-retrieval* rate (all gold tables retrieved) come from `eval/rag_retrieval_summary.csv`.

Hybrid (RRF over BM25 and dense full rankings) wins at every k . BM25 alone catches up at $k=5$ and slightly beats dense (0.94 vs 0.85 mean recall), which reflects the lexical overlap

Table 3: Retrieval-only on NBA test ($n = 50$). Mean recall@ k and rate of “perfect retrieval” (all gold tables in top- k).

Backend	k	Mean recall	Perfect rate
Dense	1	0.17	0.14
Dense	3	0.74	0.72
Dense	5	0.85	0.84
BM25	1	0.24	0.24
BM25	3	0.44	0.42
BM25	5	0.94	0.94
Hybrid	1	0.37	0.36
Hybrid	3	0.92	0.90
Hybrid	5	0.99	0.98

between question text (e.g. “free throw percentage in playoffs”) and column names (e.g. `ft_pct`, `season_type`); dense overweights paraphrases and misses these shortcuts. The interesting consequence is that *retrieval quality is essentially solved at $k \geq 3$ by hybrid*, which sets up the next section.

5.4 End-to-end RAG: retrieval is necessary but not sufficient

Table 4 reports execution accuracy for all 2 checkpoints $\times$ 3 backends $\times$ 3 k end-to-end runs (`eval/rag_e2e_summary.csv`). Compare each row against the oracle-tables upper bound for the same checkpoint (`lora_codet5p-220m_r16_bs4`: ~ 0.10 ; `lora_codet5p-220m_r16_nba_nall_s42`: **0.46**).

Table 4: End-to-end RAG: NBA test execution accuracy by checkpoint, retriever, and k . Oracle-tables upper bounds: ~ 0.10 (Spider-only), **0.46** (NBA-adapted).

Checkpoint	Backend	k	Exec acc.	Exact
<code>lora_codet5p-220m_r16_nba_nall_s42</code> (NBA-adapted)	Dense	1	0.08	0.06
	Dense	3	0.08	0.08
	Dense	5	0.08	0.08
	BM25	1	0.10	0.04
	BM25	3	0.14	0.12
	BM25	5	0.14	0.12
	Hybrid	1	0.10	0.08
	Hybrid	3	0.14	0.12
	Hybrid	5	0.12	0.12
<code>lora_codet5p-220m_r16_bs4</code> (Spider-only)	Dense	1	0.06	0.00
	Dense	3	0.08	0.00
	Dense	5	0.08	0.00
	BM25	1	0.06	0.00
	BM25	3	0.08	0.00
	BM25	5	0.08	0.00
	Hybrid	1	0.08	0.00
	Hybrid	3	0.08	0.00
	Hybrid	5	0.10	0.00

Two observations are worth emphasising:

1. **Adaptation $\gg$ retrieval method.** For the NBA-adapted checkpoint, switching from the worst RAG cell (dense, $k = 1$, 0.08) to the best (BM25 or hybrid, $k = 3$, 0.14) gains +0.06.

For the Spider-only checkpoint, the best RAG cell is 0.10. The total spread from *any* RAG configuration is small compared to the $0.46 - 0.14 = 0.32$ gap to the oracle upper bound on the adapted model.

2. **High recall does not translate.** Hybrid at $k = 5$ achieves *mean recall* 0.99 and *perfect retrieval* 0.98 on test (Table 3), but exec accuracy is only 0.12 for the adapted model and 0.10 for the Spider model. The bottleneck is no longer “missing tables.”

Figure 3 plots mean retrieval recall against execution accuracy with one panel per checkpoint, and Figure 4 shows the backend comparison at $k = 3$. The two figures together summarise the “RAG breaks” message: recall climbs cleanly into the high-0.9s, but exec accuracy plateaus near the bottom.

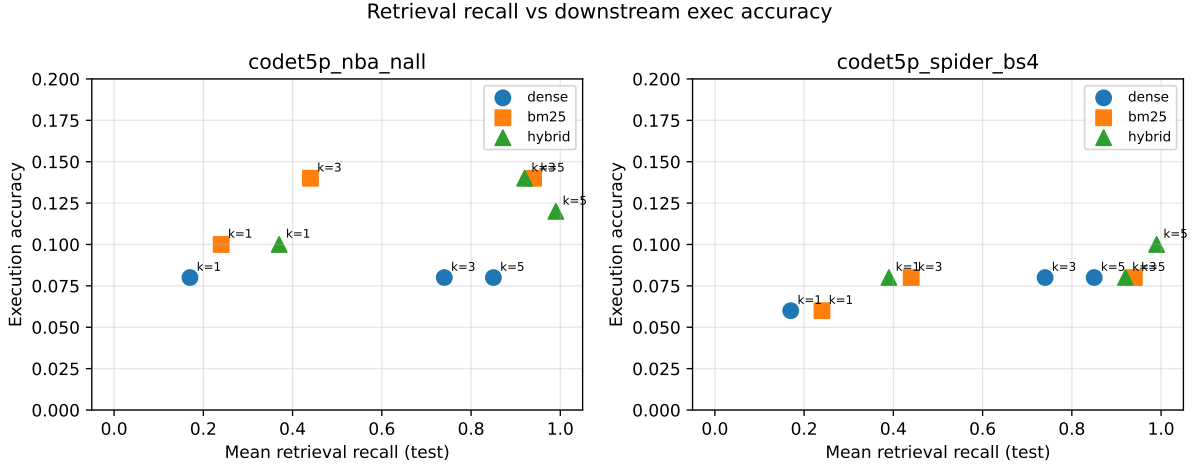


Figure 3: Per-run mean retrieval recall (x) vs NBA test execution accuracy (y), faceted by checkpoint. Marker shape encodes retrieval backend; annotations show k . Even when hybrid reaches near-perfect recall on the right edge of each panel, downstream exec accuracy stays in the 0.08–0.14 band, far from the oracle upper bound (0.46 for the NBA-adapted panel).

5.5 Recall bins: a threshold effect, not a smooth gradient

Table 5 stratifies test examples by per-question retrieval recall into three bins (< 0.4 , $0.4\text{--}0.7$, > 0.7) and reports execution rate within each bin, on the NBA-adapted checkpoint (`eval/rag_recall_bins_summ`).

Table 5: NBA-adapted checkpoint: execution rate stratified by per-example retrieval recall, by retrieval backend and k . Empty cells indicate zero examples in that bin (‘–’).

Backend	k	recall < 0.4		recall $0.4\text{--}0.7$		recall > 0.7	
		n	exec	n	exec	n	exec
Dense	1	40	0.00	3	0.00	7	0.57
Dense	3	12	0.00	2	0.00	36	0.11
Dense	5	7	0.00	1	0.00	42	0.10
BM25	1	38	0.00	0	–	12	0.42
BM25	3	27	0.00	2	0.00	21	0.33
BM25	5	3	0.00	0	–	47	0.15
Hybrid	1	31	0.00	1	0.00	18	0.28
Hybrid	3	3	0.00	2	0.00	45	0.16
Hybrid	5	0	–	1	0.00	49	0.12

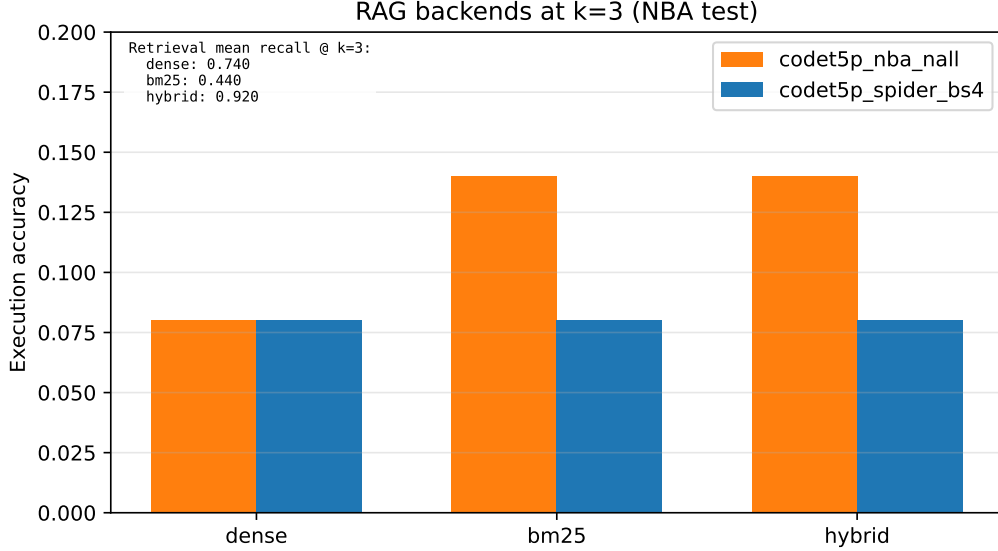


Figure 4: NBA test execution accuracy at $k = 3$ by retrieval backend, grouped by checkpoint. The retrieval-mean-recall annotations match Table 3 (BM25 0.44, dense 0.74, hybrid 0.92 at $k = 3$).

There are two regularities. First, the low-recall bin (recall < 0.4) is essentially *always 0% exec* – if the retriever misses gold tables, generation cannot recover. Second, the high-recall bin (recall > 0.7) drops noticeably as k grows: BM25 at $k = 1$ on the perfectly retrieved subset is 0.42, but at $k = 5$ it falls to 0.15. The drop is a *distractor effect*: more candidate tables in the prompt (some plausible but wrong, e.g. `game_summary` vs `game`) push the decoder toward the wrong join. This is exactly the threshold effect hypothesised in the project todo and is the cleanest piece of evidence for the “RAG breaks the system” framing.

5.6 Failure-case sample

Table 6 samples 10 failed examples from `lora_codet5p-220m_r16_nba_nall_s42` with dense $k = 3$ (`internal_docs/rag_failure_cases.md`), categorised as *retrieval_miss* (gold tables not all retrieved) vs *gen_error* (perfect retrieval, wrong SQL). The split is exactly 5/5 in this small sample, mirroring the bin analysis: half the failures are upstream of generation, half are not.

5.7 Error taxonomy

`src/error_analysis.py` runs a rule-based classifier over the predicted SQL and writes `eval/error_analysis_`. Table 7 compares oracle vs RAG ($k = 3$, dense) on `lora_codet5p-220m_r16_nba_nall_s42`.

The shift is striking: under oracle schema, residual errors are dominated by *value* (literal mismatches, e.g. `'Lakers'` vs `'Los Angeles Lakers'`, `season_id = '22022'` vs `= 22022`, etc.) and *structural* (missing `ORDER BY ... LIMIT 1`). Under dense RAG $k=3$, the dominant class shifts to *schema_linking*: the model gets a few candidate tables and writes against the wrong one. This pattern is consistent with the recall-bin and failure-case views.

6 Analysis and Discussion

6.1 What worked

1. **In-domain adaptation dominated the comparison.** The Spider $\rightarrow$ NBA gap is enormous and is closed almost entirely by NBA data, not by retrieval, model size within T5-BASE

Table 6: Sampled failures from `lora_codet5p-220m_r16_nba_nall_s42` with dense $k = 3$. Truncated for space. Full table in `internal_docs/rag_failure_cases.md`.

#	category	question (trunc.)	retrieved	recall	predicted SQL (trunc.)
1	retrieval_miss	active players who were the #1 overall pick	draft_history, inactive_players, player	0.5	SELECT * FROM draft_history WHERE overall_pick = 1
2	retrieval_miss	types of games recorded	draft_history, game_summary, other_stats	0.0	SELECT DISTINCT draft_type FROM draft_history
3	retrieval_miss	home team scoring at least ... in 22-23 RS	other_stats, line_score, game_summary	0.0	SELECT COUNT(*) FROM other_stats WHERE pts_2nd_chance_home >= 130
4	retrieval_miss	max single-game home rebound total in 22-23 RS	other_stats, draft_combine, inactive	0.0	SELECT MAX(team_rebounds_home) FROM other_stats ...
5	retrieval_miss	both teams scoring ≥ 120 in 22-23 RS	other_stats, line_score, game_summary	0.0	SELECT COUNT(*) FROM line_score WHERE pts_away >= 120
6	gen_error	how many active players	inactive_players, player, play_by_play	1.0	SELECT COUNT(*) FROM player (missing is_active = 1)
7	gen_error	home games Lakers played in 22-23 RS	game_info, game_summary, game	1.0	...WHERE team_name_home = 'Lakers' AND season_id = '22022' ... (gold uses 'Los Angeles Lakers')
8	gen_error	team with highest avg home pts in 22-23 RS	other_stats, line_score, game	1.0	SELECT team_name_home, AVG(pts_home) ... (missing ORDER BY ...LIMIT 1)
9	gen_error	teams with home win rate $> 70\%$	other_stats, line_score, game	1.0	...team_wins_losses_home ... (column does not exist)
10	gen_error	best 3-pt % at home in playoffs	other_stats, line_score, game	1.0	...WHERE season_id = 'Playoff' ... (wrong literal vs season_type)

Table 7: Error category rates on NBA test ($n=50$) for the NBA-adapted checkpoint. “correct” = exec match; remaining categories are mutually exclusive heuristics.

Mode	correct	schema_linking	structural	value
oracle	0.46	0.06	0.18	0.30
RAG $k=3$ (dense)	0.08	0.78	0.06	0.08

family, or LoRA rank. CODET5+ + LoRA $r=16 + n_{\text{train}}$ all takes test exec from 0.10 to 0.46.

- PEFT made the experiment grid feasible.** On a single 3080 it was practical to run rank sweeps, the few-shot curve, and the 18 RAG e2e runs without queueing.
- LoRA-only NBA adaptation fixed catastrophic forgetting.** Switching from “merge then full FT” to “train adapter, merge after” restored the few-shot curve from 0.00 exec to the values in Table 2.
- Retrieval is solvable for this benchmark.** Hybrid (RRF) at $k \in \{3, 5\}$ achieves ≥ 0.92 mean recall and ≥ 0.90 perfect retrieval (Table 3); we are no longer retrieval-bottlenecked at the IR layer.

6.2 What did not work

1. **High retrieval recall did not propagate to exec accuracy.** Hybrid $k=5$ at recall 0.99 still scored 0.12 exec on the adapted model – a $4\times$ gap to the oracle. The bottleneck is downstream schema-conditioned generation, not retrieval.
2. **More candidate tables actively hurt.** The recall-bin analysis (Section 5.5) shows the high-recall subset’s exec rate *decreasing* as k grows for BM25 and hybrid (BM25: $0.42 \rightarrow 0.33 \rightarrow 0.15$). Distractor tables in the prompt push the decoder toward the wrong join.
3. **Spider exact match is harsh and execution accuracy was not run on Spider.** Both directions miss something: Spider exact match is conservative, and execution-grounded comparison on Spider would need per-database SQLite handling we did not wire in.

6.3 Failure modes

The error taxonomy (Section 5.7) and the sampled failures (Section 5.6) converge on four recurring classes: schema linking (wrong table or column), structural errors (missing clauses, malformed joins), value-grounding errors (wrong literals such as the ‘22022’ season encoding), and aggregation/ordering mistakes (missing `ORDER BY ... LIMIT 1`).

6.4 Limitations

- Most NBA runs are single-seed; while Table 2 reports bootstrap CIs, true seed variance is not separated from sampling variance.
- The NBA QA set is hand-authored (~ 200 questions); annotation noise and question-style bias are real.
- RAG uses fixed `all-MiniLM-L6-v2` + BM25 + RRF without a cross-encoder re-ranker; BM25 tokenisation is intentionally simple.
- The Spider control table is exact match only.

7 Conclusion

This Applied + Training project delivers (1) a working HoopQL pipeline with three schema modes and a Gradio demo, and (2) a controlled study of training regimes and retrieval backends on a fixed NBA hold-out. The dominant finding is that in-domain adaptation data, not retrieval sophistication, drives execution accuracy on this benchmark: `CODET5+` + `LoRA` + full NBA adaptation reaches 0.46 exec under oracle schema, whereas the best end-to-end RAG configuration (BM25 or hybrid at $k=3$) tops out near 0.14 despite mean retrieval recall of 0.92 at the same k . The recall-bin analysis explains why: the low-recall bin is unrecoverable, and the high-recall bin actively degrades as k grows due to distractor tables in the prompt.

For follow-on work, the priorities suggested by these results are schema-conditioned decoding (table-aware constrained decoding to suppress distractor columns), a cross-encoder re-ranker on top of hybrid RRF, and a multi-seed run of the few-shot curve to convert the bootstrap CIs into seed-aware variance estimates. The full reproducibility stack (`src/rag.py`, `src/evaluate.py`, `scripts/run_rag_retrieval_ablation.py`, `scripts/plot_rag_ablation.py`, and `src/build_results_rep`) is in the repository, and every table and figure in this write-up is regenerable from the contents of `eval/` via two commands documented in the README.

Use of LLM assistants. LLM-based coding assistants (Claude, GPT) were used during development for code refactoring, debugging the LoRA-adapter merge bug, and drafting boilerplate LaTeX. All experimental decisions, runs, analyses, and final prose are my own; I can explain any line of code or any number in this report.

References

- [1] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *EMNLP*, 2018.
- [2] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022.
- [3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*, 2023.
- [4] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *NeurIPS*, 2020.
- [5] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *JMLR*, 21(140):1–67, 2020.
- [6] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *EMNLP*, 2021.
- [7] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [8] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods. In *SIGIR*, 2009.
- [9] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*, 2019.
- [10] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2019.